🟢 Aula 9 — Gerenciamento de Estado (Pinia) (4h)

🎯 Objetivos da aula

- Compreender **estado global** e quando utilizá-lo.
- Comparar **Pinia vs Vuex** e entender o padrão atual do Vue 3.
- Criar uma **store Pinia** bem estruturada.
- **Compartilhar dados entre componentes** sem prop drilling.
- **Refatorar estado local → global** seguindo boas práticas.

---

1) Teoria (aprox. 1h30)

1.1 O que é Estado Global

Estado global é o **estado compartilhado** por múltiplos componentes/rotas:

- Usuário logado
- Lista de usuários/produtos
- Preferências (tema, idioma)
- Cache de dados da API

Problema sem estado global: *prop drilling* (props passando por vários níveis).

1.2 Pinia vs Vuex

Critério	Pinia	Vuex
Status	**Oficial (Vue 3)**	Legado
API	Simples e moderna	Verbosa

Critério	Pinia	Vuex
Tipagem	Excelente (TS-friendly)	Mais complexa
Mutations	❌ (não precisa)	✅ obrigatórias
Boilerplate	Baixo	Alto

👉 **Conclusão:** projetos novos usam **Pinia**.

1.3 Conceitos do Pinia

- **Store**: módulo de estado
- **state**: dados
- **getters**: valores derivados (como computed)
- **actions**: lógica/efeitos (async)

```
state → dados getters → derivados actions → ações (sync/async)
```

---

2) Prática (aprox. 2h30) — Estado Global em ação

> 🎯 **Contexto**
> Refatorar o projeto para:

- Centralizar **usuários** em uma store
- Consumir API na store
- Usar o estado em **views diferentes**

- Remover estado duplicado dos componentes

2.1 Instalação do Pinia

```
npm install pinia
```

2.2 Registro do Pinia no projeto

src/main.js

```
import { createApp } from 'vue' import { createPinia } from 'pinia' import App from './App.vue' import router from './router'
const app = createApp(App) // cria a instância do Pinia const pinia = createPinia() app .use(pinia) // registra Pinia
.use(router) .mount('#app')
```

2.3 Estrutura de pastas (padrão)

```
src/
 ├─ stores/
 │   └─ userStore.js
 ├─ views/
 │   ├─ UsersView.vue
 │   └─ UserDetailView.vue
 ├─ services/
```

```
|   └─ api.js
└─ App.vue
```

---

3) Criando a Store (estado global)

3.1 `src/stores/userStore.js`

```
import { defineStore } from 'pinia' import api from '../services/api' export const useUserStore = defineStore('userStore', { // ====================== // STATE (dados globais) // ====================== state: () => ({ usuarios: [], loading: false, erro: '' }), // ====================== // GETTERS (derivados) // ====================== getters: { totalUsuarios: (state) => state.usuarios.length, usuarioPorId: (state) => { return (id) => state.usuarios.find(u => u.id === Number(id)) } }, // ====================== // ACTIONS (lógica) // ====================== actions: { async carregarUsuarios() { this.loading = true this.erro = '' try { const response = await api.get('/users') this.usuarios = response.data } catch (e) { this.erro = 'Erro ao carregar usuários.' } finally { this.loading = false } } } })
```

✅ **Boas práticas aplicadas**

- API chamada **na store**, não no componente
- `getters` para valores derivados
- Estado único e reutilizável

---

4) Usando a Store nos Componentes

4.1 `views/UsersView.vue` — Lista usando estado global

```
<template> <section> <h2>Usuários</h2> <p>Total: {{ userStore.totalUsuarios }}</p> <p v-if="userStore.loading">Carregando...</p> <p v-else-if="userStore.erro" class="error"> {{ userStore.erro }} </p> <ul v-else> <li v-for="u in userStore.usuarios"
```

```
:key="u.id"> {{ u.name }} <router-link :to="`/usuarios/${u.id}`"> Detalhes </router-link> </li> </ul> <button
@click="userStore.carregarUsuarios"> Recarregar </button> </section> </template> <script> import { useUserStore } from
'../stores/userStore' export default { name: 'UsersView', setup() { // acesso à store const userStore = useUserStore() // carrega
dados ao entrar na página if (userStore.usuarios.length === 0) { userStore.carregarUsuarios() } return { userStore } } }
</script> <style scoped> .error { color: #c62828; } </style>
```

4.2 `views/UserDetailView.vue` — Detalhe via getter

```
<template> <section> <h2>Detalhes do Usuário</h2> <p v-if="!usuario">Usuário não encontrado.</p> <div v-else> <p><strong>Nome:
</strong> {{ usuario.name }}</p> <p><strong>Email:</strong> {{ usuario.email }}</p> <p><strong>Telefone:</strong> {{
usuario.phone }}</p> </div> </section> </template> <script> import { useUserStore } from '../stores/userStore' export default {
name: 'UserDetailView', props: ['id'], setup(props) { const store = useUserStore() const usuario = store.usuarioPorId(props.id)
return { usuario } } } </script>
```

---

5) Refatoração: antes × depois

❌ Antes (estado local duplicado)

```
data() { return { usuarios: [] } }
```

✅ Depois (estado global)

```
const userStore = useUserStore()
```

✔ Menos código
✔ Menos bugs

✔️ Estado único

---

6) O que foi consolidado

✔️ Estado global

- Centralizado
- Reutilizável
- Previsível

✔️ Pinia

- API simples
- Integração perfeita com Vue 3

✔️ Refatoração correta

- Componentes focados em UI
- Store focada em dados e regras

---

7) Atividade prática da aula

🎯 Desafio

1. Criar getter `usuariosComEmailBiz`
   - Filtrar emails `@biz`
2. Criar action `limparUsuarios`
3. Usar o getter em uma nova view `/usuarios-biz`

💡 Dica:

```
usuariosComEmailBiz: (state) => state.usuarios.filter(u => u.email.endsWith('.biz'))
```

---

8) Checklist de encerramento

- Instalou Pinia
- Criou store
- Usou getters
- Centralizou API
- Refatorou estado local